

GitHub Copilot Custom Instructions for 3yeses

Project Overview

3yeses is a subscription-based talent marketplace platform built with Next.js 14 App Router, connecting creative professionals (actors, musicians, models, voice artists, dancers) with opportunities. The platform features multi-language support, advanced search/filtering, analytics, subscription management via Stripe, and a full admin dashboard.

Key Stats:

- 501 talent profiles across 21 categories
- 2,885 portfolio items (images, videos, audio)
- PostgreSQL with Prisma ORM (20 database models)
- Single subscription tier: Standard (£10/6 months, £20/year)
- 11 language support via next-intl
- Port 3002 for local development

Test Account:

- Email: talent1@test.com
- Password: Test123!
- Profile ID: cmlbas5620002vvc49snthhi4

Tech Stack

- **Framework:** Next.js 14.2.33 with App Router
- **Language:** TypeScript (strict mode)
- **Database:** PostgreSQL with Prisma ORM 6.16.2
- **Styling:** Tailwind CSS
- **Authentication:** JWT with bcryptjs (12 rounds)
- **Payments:** Stripe (subscriptions + webhooks)
- **Internationalization:** next-intl 4.3.9 (11 locales)
- **Media:** Cloudinary integration, React Player 2.16.0
- **Email:** Resend API 6.4.1
- **State Management:** SWR 2.3.4 for client-side data fetching
- **Testing:** Playwright for E2E tests
- **Analytics:** Recharts 3.3.0

Architecture Guidelines

Next.js App Router Patterns

- **Default to Server Components:** Do not add 'use client' unless hooks, event handlers, or browser APIs are required
- **API Routes:** Use `app/api/[...]/route.ts` pattern with GET/POST/PUT/DELETE handlers
- **Data Fetching:**
 - Server Components: Direct Prisma queries or async fetch
 - Client Components: SWR hooks with /api endpoints
- **Layouts:** Leverage `layout.tsx` for shared UI, avoid prop drilling
- **Loading States:** Use `loading.tsx` for automatic Suspense boundaries
- **Error Handling:** Create `error.tsx` for boundary error handling
- **Middleware:** Apply auth checks and locale routing via `middleware.ts`

File Structure Conventions

app/
[locale]/ # Internationalized routes
api/ # API routes (auth, talent, subscriptions, webhooks)
admin/ # Admin dashboard routes
dashboard/ # Talent dashboard routes
talent/[id]/ # Dynamic talent profile pages
components/
admin/ # Admin-specific components
dashboard/ # Dashboard widgets
auth/ # Auth forms
shared/ # Reusable components (TalentCard, FilterSidebar)
lib/
services/ # Business logic (AuthService, analytics)
hooks/ # React hooks
prisma.ts # Prisma client singleton
prisma/
schema.prisma # Database schema (20 models)
seed.ts # Seed data
scripts/ # Utility scripts (generate-500-talents.mjs)
messages/ # i18n translation files (en.json, de-DE.json, etc.)

Import Rules

- **Server-only modules:** Import Prisma and AuthService directly (`import { prisma } from '@lib/prisma'`)
- **Client-safe modules:** Can use barrel exports (`import { useTalent } from '@lib'`)
- **Avoid circular dependencies:** Keep `lib/services` separate from `components`

Database Schema (Prisma)

Subscription Model (Critical)

```
model Subscription {
  id String @id @default(cuid())
  userId String
  plan SubscriptionPlan @default(STANDARD)
  status SubscriptionStatus @default(ACTIVE)
  stripeSubscriptionId String? @unique
  stripeCustomerId String?
  stripePriceId String?
  currentPeriodStart DateTime?
  currentPeriodEnd DateTime?
  startDate DateTime
  endDate DateTime?
  cancelAt DateTime?
  canceledAt DateTime?
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  user User @relation(fields: [userId], references: [id], onDelete: Cascade)
}
```

```
enum SubscriptionPlan {
  STANDARD
}
```

```
enum SubscriptionStatus {
  ACTIVE
  CANCELED
  PAST_DUE
  UNPAID
  TRIALING
}
```

Key Models

- **User:** Authentication (TALENT/ADMIN roles), email verification, parental consent (ages 13-15)
- **TalentProfile:** Extended profile (15+ physical attributes, skills, languages, portfolio)
- **PortfolioItem:** Multi-media (IMAGE, VIDEO, AUDIO, DOCUMENT, LINK)
- **Subscription & Payment:** Stripe integration for Standard plan
- **Category & Subcategory:** 21 categories with 100+ subcategories
- **Review, Comment, Like:** Social engagement
- **Report:** Content moderation
- **ProfileView, PortfolioView, SearchAppearance, ProfileStats:** Analytics tracking

Database Best Practices

- Use Prisma client (prisma.user.findUnique()) - never raw SQL
- Include error handling with try/catch for all queries
- Use transactions for multi-model operations
- Leverage include and select to optimize queries
- Apply indexes for search fields (gender, ethnicity, location, rating, skills)

TypeScript Guidelines

Strict Mode Requirements

- Enable strict: true in tsconfig.json
- No any types - use unknown and type guards if needed
- Explicit return types on all functions
- Prefer interface over type for object shapes
- Use Prisma-generated types (Prisma.UserCreateInput, TalentProfile)

Type Patterns

```
// API Response Types
interface ApiResponse<T> {
  data: T | null;
  error: { code: string; message: string } | null;
  meta?: { timestamp: number };
}

// Zod Validation (API routes)
import { z } from 'zod';
const subscriptionSchema = z.object({
  plan: z.enum(['STANDARD']),
  priceId: z.string(),
});

// Extend Prisma Types
import { User, Subscription } from '@prisma/client';
type UserWithSubscription = User & { subscription: Subscription | null };

// Async Function Types
async function getSubscription(userId: string): Promise<Subscription | null> {
  return await prisma.subscription.findUnique({ where: { userId } });
}
```

Naming Conventions

- **Components:** PascalCase (TalentCard.tsx)
- **Utilities:** camelCase (formatCurrency.ts)
- **Types:** PascalCase (UserWithSubscription)
- **API Routes:** kebab-case folders (/api/auth/verify-email/route.ts)
- **Database fields:** camelCase in schema, snake_case in DB

Security Standards

Authentication & Authorization

- **JWT tokens:** httpOnly cookies, 7-day expiry, refresh tokens (30 days)
- **Password hashing:** bcryptjs with 12 rounds minimum
- **Email verification:** Required before full access, 24-hour token expiry
- **Session validation:** Verify JWT on every API request and middleware
- **Role-based access:** TALENT vs ADMIN, gate admin routes with middleware

- **Parental consent:** Required for users aged 13-15 before account activation

API Security

```
// Example: Secure API Route
import { verifyAuth } from '@lib/services/auth/auth-service';
import { prisma } from '@lib/prisma';

export async function POST(request: Request) {
  // 1. Verify authentication
  const user = await verifyAuth(request);
  if (!user) {
    return NextResponse.json(
      { data: null, error: { code: 'UNAUTHORIZED', message: 'Authentication required' } },
      { status: 401 }
    );
  }

  // 2. Validate input with Zod
  const body = await request.json();
  const validation = subscriptionSchema.safeParse(body);
  if (!validation.success) {
    return NextResponse.json(
      { data: null, error: { code: 'VALIDATION_ERROR', message: validation.errorMessage } },
      { status: 400 }
    );
  }

  // 3. Perform authorized operation
  try {
    const result = await prisma.subscription.create({ data: validation.data });
    return NextResponse.json({ data: result, error: null });
  } catch (error) {
    return NextResponse.json(
      { data: null, error: { code: 'SERVER_ERROR', message: 'Operation failed' } },
      { status: 500 }
    );
  }
}
```

Stripe Security

- **Webhook verification:** Always verify stripe-signature header
- **Price IDs:** Only pass Stripe price IDs from client, never amounts
- **Customer IDs:** Store stripeCustomerId in database, verify ownership
- **Idempotency:** Use idempotency keys for payment operations
- **Metadata:** Store userId in Stripe metadata for reconciliation

```
// Webhook Verification
import { headers } from 'next/headers';
import Stripe from 'stripe';

const stripe = new Stripe(process.env.STRIPE_SECRET_KEY!);
```

```
export async function POST(request: Request) {
  const body = await request.text();
  const signature = headers().get('stripe-signature')!;
```

```
  try {
    const event = stripe.webhooks.constructEvent(
      body,
      signature,
      process.env.STRIPE_WEBHOOK_SECRET!
    );
```

```
    // Handle event types
    if (event.type === 'checkout.session.completed') {
      // Update subscription status
    }
  }
```

```
  } catch (err) {
    return NextResponse.json({ error: 'Webhook signature verification failed' }, { status: 400 });
  }
}
```

File Upload Security

- **Cloudinary signed uploads:** Generate signatures server-side
- **File type validation:** Images (jpg, png, webp), videos (mp4, mov), audio (mp3, wav)
- **Size limits:** 10MB for images, 100MB for videos, 20MB for audio
- **Content scanning:** Cloudinary moderation for inappropriate content
- **Direct uploads:** Use Cloudinary widget with signed upload presets

Data Protection

- **Sensitive data:** Never log passwords, tokens, or payment details
- **CORS:** Restrict API origins to production domains
- **Rate limiting:** Implement with Upstash or middleware (10 req/min for auth, 100/day for uploads)
- **CSRF tokens:** Use Next.js built-in CSRF protection or NextAuth
- **Input sanitization:** Escape user-generated content before rendering

Subscription & Payment Logic

Standard Plan Features

Pricing:

- 6 months: £10 (recurring)
- 12 months: £20 (2 × 6-month periods or yearly product)

Features:

- Full profile customization

- Unlimited portfolio uploads (images, videos, audio)
- Direct messaging (if allowDirectContact enabled)
- Priority search ranking
- Advanced analytics dashboard
- Profile visibility controls

Feature Gating

```
// Check subscription status
function hasActiveSubscription(user: UserWithSubscription): boolean {
  return user.subscription?.status === 'ACTIVE';
}

// Example: Portfolio upload limit
async function canUploadPortfolio(userId: string): Promise<boolean> {
  const user = await prisma.user.findUnique({
    where: { id: userId },
    include: { subscription: true },
  });

  if (!user) return false;

  // Standard plan: unlimited uploads
  if (hasActiveSubscription(user)) return true;

  // No subscription: limit to 3 items
  const portfolioCount = await prisma.portfolioItem.count({
    where: { talentProfile: { userId } },
  });

  return portfolioCount < 3;
}
```

Stripe Checkout Flow

1. User clicks "Subscribe with Stripe" button
2. POST /api/subscribe creates Stripe Checkout session
3. Redirect to Stripe hosted checkout page
4. User completes payment
5. Stripe webhook /api/webhook confirms payment
6. Update subscription record with ACTIVE status
7. Redirect user to dashboard with full access

Internationalization (i18n)

next-intl Configuration

- **Supported locales:** en, en-gb, de-DE, es-ES, fr-FR, it-IT, ja-JP, pt-PT, ru-RU, zh-CN, ar
- **Default locale:** en
- **RTL support:** Arabic (ar) locale
- **Translation files:** messages/[locale].json
- **Routing:** Dynamic [locale] segment in app directory

Translation Patterns

```
// Server Component
import { useTranslations } from 'next-intl';
import { getTranslations } from 'next-intl/server';
```

```
// Client Component
export default function Component() {
  const t = useTranslations('Dashboard');
  return
```

```
    {t('welcome')}
```

```
  ;
}
```

```
// Server Component
export default async function Page() {
  const t = await getTranslations('Dashboard');
  return
```

```
    {t('welcome')}
```

```
  ;
}
```

Translation Keys Structure

```
{
  "Dashboard": {
    "welcome": "Welcome back",
    "subscription": {
      "active": "Your subscription is active",
      "upgrade": "Upgrade to Standard"
    }
  },
  "Pricing": {
    "standard": {
      "title": "Standard Access",
      "price": "£10 / 6 months",
      "features": {
        "customization": "Full Profile Customization",
        "uploads": "Unlimited Portfolio Uploads"
      }
    }
  }
}
```


Code Style & Patterns

React Component Patterns

// Server Component (default)

```
import { prisma } from '@lib/prisma';
```

```
export default async function TalentPage({ params }: { params: { id: string } }) {  
  const talent = await prisma.talentProfile.findUnique({  
    where: { id: params.id },  
    include: { user: true, portfolio: true },  
  });
```

```
  if (!talent) {  
    return  
    Talent not found  
  }  
}
```

```
  return ;  
}
```

// Client Component (with 'use client')

'use client';

```
import { useState } from 'react';
```

```
import useSWR from 'swr';
```

```
export default function TalentList() {  
  const [filters, setFilters] = useState({});  
  const { data, error } = useSWR(/api/talent/search?${new URLSearchParams(filters)});
```

```
  if (error) return  
  Failed to load  
  ;  
  if (!data) return  
  Loading...  
  ;
```

```
  return (
```

```
    {data.talents.map((talent) => (
```

```
    ))}
```

```
  );  
}
```

Error Handling

```
// API Route Error Handling
try {
  const result = await performOperation();
  return NextResponse.json({ data: result, error: null });
} catch (error) {
  console.error('Operation failed:', error);

  if (error instanceof Prisma.PrismaClientKnownRequestError) {
    if (error.code === 'P2002') {
      return NextResponse.json(
        { data: null, error: { code: 'DUPLICATE', message: 'Record already exists' } },
        { status: 409 }
      );
    }
  }

  return NextResponse.json(
    { data: null, error: { code: 'SERVER_ERROR', message: 'Internal server error' } },
    { status: 500 }
  );
}
```

Async/Await Patterns

- Always use `async/await` over `.then()` chains
- Add `try/catch` for error handling
- Use `Promise.all()` for parallel operations
- Avoid nested awaits when operations are independent

Testing Standards

Playwright E2E Tests

- **Location:** `tests/e2e/`
- **Key flows:**
 - `signup-simple.spec.ts` - Basic signup flow
 - `signup-parental-consent.spec.ts` - Parental consent for ages 13-15
- **Commands:**
 - `npm run test` - Run all tests
 - `npm run test:ui` - Playwright UI mode
 - `npm run test:headed` - Run with browser visible

Test Patterns

```
// Example E2E Test
import { test, expect } from '@playwright/test';

test('talent can subscribe to Standard plan', async ({ page }) => {
  // Login
  await page.goto('http://localhost:3002/en/auth/login');
```

```
await page.fill('input[name="email"]', 'talent1@test.com');
await page.fill('input[name="password"]', 'Test123!');
await page.click('button[type="submit"]');

// Navigate to pricing
await page.goto('http://localhost:3002/en/pricing');
await page.click('text=Subscribe with Stripe');

// Verify Stripe Checkout redirect
await expect(page).toHaveURL(/checkout.stripe.com/);
});
```

Environment Variables

Required Variables

Database

DATABASE_URL=postgresql://user:password@localhost:5432/3yeses

Authentication

JWT_SECRET=your-secret-key
JWT_EXPIRES_IN=7d
JWT_REFRESH_EXPIRES_IN=30d

Email

RESEND_API_KEY=re_XXXXXXXXXXXX
[EMAIL_FROM=noreply@3yeses.com](mailto:noreply@3yeses.com)

Media

CLOUDINARY_CLOUD_NAME=xxxxxx
CLOUDINARY_API_KEY=xxxxxx
CLOUDINARY_API_SECRET=xxxxxx

Payments

STRIPE_PUBLIC_KEY=pk_test_XXXXXXXXXXXX
STRIPE_SECRET_KEY=sk_test_XXXXXXXXXXXX
STRIPE_WEBHOOK_SECRET=whsec_XXXXXXXXXXXX

Stock Media APIs (for data generation)

PEXELS_API_KEY=xxxxxx
UNSPLASH_ACCESS_KEY=xxxxxx
PIXABAY_API_KEY=xxxxxx
FREESOUND_API_KEY=xxxxxx
YOUTUBE_API_KEY=xxxxxx

Development Scripts

Development

npm run dev # Start dev server (port 3002)
npm run dev:clean # Kill servers and start fresh
npm run build # Production build
npm run start # Start production server
npm run lint # Run ESLint
npm run lint:fix # Fix linting issues

Database

npx prisma db push # Push schema changes
npx prisma db seed # Seed database
npx prisma generate # Generate Prisma client
npx prisma studio # Visual DB editor
npx prisma format # Format schema file

Data Generation

node scripts/generate-500-talents.mjs # Generate 500 talents
node scripts/count-talents.mjs # Count talents

Testing

npm run test # Run Playwright tests
npm run test:ui # Playwright UI mode
npm run test:headed # Run with browser visible

Common Patterns & Snippets

Middleware Pattern (Auth Check)

```
// middleware.ts
import { NextResponse } from 'next/server';
import type { NextRequest } from 'next/server';
import { verifyToken } from '@lib/services/auth/auth-service';
```

```
export async function middleware(request: NextRequest) {
  const token = request.cookies.get('token')?.value;

  if (!token) {
    return NextResponse.redirect(new URL('/auth/login', request.url));
  }
}
```

```
try {
  const user = await verifyToken(token);
```

```
  // Check subscription for premium routes
  if (request.nextUrl.pathname.startsWith('/dashboard/analytics')) {
    if (!user.subscription?.status === 'ACTIVE') {
      return NextResponse.redirect(new URL('/pricing', request.url));
    }
  }

  return NextResponse.next();
}
```

```
  } catch {
    return NextResponse.redirect(new URL('/auth/login', request.url));
  }
}
```

```
export const config = {
  matcher: ['/dashboard/:path*', '/admin/:path*'],
};
```

SWR Data Fetching

```
// hooks/useTalent.ts
import useSWR from 'swr';

const fetcher = (url: string) => fetch(url).then((res) => res.json());

export function useTalent(id: string) {
  const { data, error, mutate } = useSWR(`/api/talent/${id}`, fetcher, {
    revalidateOnFocus: false,
    dedupingInterval: 60000, // 1 minute
  });

  return {
    talent: data?.data,
    isLoading: !error && !data,
    isError: error,
    mutate,
  };
}
```

Subscription Check Utility

```
// lib/utls/subscription.ts
import { Subscription, SubscriptionStatus } from '@prisma/client';

export function hasActiveSubscription(subscription: Subscription | null): boolean {
  if (!subscription) return false;
  return subscription.status === SubscriptionStatus.ACTIVE;
}

export function isSubscriptionExpired(subscription: Subscription | null): boolean {
  if (!subscription || !subscription.currentPeriodEnd) return true;
  return new Date() > subscription.currentPeriodEnd;
}

export function getSubscriptionFeatures(subscription: Subscription | null) {
  const hasActive = hasActiveSubscription(subscription);

  return {
    unlimitedUploads: hasActive,
    prioritySearch: hasActive,
    advancedAnalytics: hasActive,
    directMessaging: hasActive,
    profileCustomization: hasActive,
    maxPortfolioItems: hasActive ? Infinity : 3,
  };
}
```

Performance Optimization

Image Optimization

- Use Next.js `<Image>` component for all images
- Cloudinary transformations for thumbnails and responsive images
- Lazy load portfolio items with intersection observer
- WebP format with fallbacks

Database Query Optimization

- Use `select` to limit fields returned
- Apply `take` and `skip` for pagination (limit 20 per page)
- Leverage Prisma indexes on frequently queried fields
- Use `findUnique` over `findMany` when possible
- Batch queries with `findMany` + `where: { id: { in: [...] } }`

Caching Strategy

- SWR for client-side data caching (1-minute deduplication)
- Next.js Route Cache for static pages
- Prisma connection pooling (limit 10 connections)
- Redis/Upstash for session storage (future enhancement)

Accessibility Standards

- Use semantic HTML elements (<header>, <nav>, <main>, <footer>)
- ARIA labels for interactive elements
- Keyboard navigation support (tab order, focus states)
- Color contrast ratio 4.5:1 for normal text, 3:1 for large text
- Alt text for all images
- Form labels and error messages
- Focus indicators visible

Known Issues & Workarounds

YouTube API Rate Limiting

- Script generate-500-talents.mjs may hit rate limits
- Graceful degradation: Falls back to placeholder videos
- No action required - script continues with other sources

Unsplash API Keywords

- Some specific keywords may fail (e.g., niche subcategories)
- Fallback: Uses Pexels or generic search terms
- No impact on data generation completion

Browser Storage Security

- Never use localStorage, sessionStorage, or document.cookie directly
- Throws SecurityError in sandbox environments
- Use server-side state or httpOnly cookies via API routes

Prohibited Patterns

- ✗ Raw SQL queries (use Prisma only)
- ✗ any types without justification
- ✗ Client-side secrets or API keys
- ✗ Unvalidated user input in database queries
- ✗ Missing error handling on async operations
- ✗ Hardcoded prices or feature flags (use Stripe/database)
- ✗ Importing server-only modules in client components
- ✗ console.log in production code (use proper logging)
- ✗ Mixing then() and async/await patterns
- ✗ Circular imports between modules

Workflow Best Practices

Pull Request Checklist

- [] Run `npm run lint:fix` and resolve all errors
- [] Run `npm run test` and ensure all tests pass
- [] Update relevant documentation if API changes
- [] Add translations for new user-facing text
- [] Test subscription feature gating for new features
- [] Verify Stripe webhook handling for payment flows
- [] Check mobile responsiveness for UI changes

Git Commit Messages

Follow conventional commits format:

feat: add subscription upgrade flow to dashboard

fix: resolve JWT token expiry handling in middleware

docs: update Copilot instructions with payment examples

test: add E2E test for parental consent flow

refactor: extract subscription utilities to shared module

Branch Naming

- feat/subscription-upgrade-ui
- fix/jwt-expiry-handling
- docs/copilot-instructions
- test/parental-consent-e2e

Support & Resources

- **GitHub Issues:** For bugs and feature requests
- **DEVDOC.md:** Detailed developer documentation
- **Prisma Schema:** `prisma/schema.prisma` (single source of truth)
- **API Docs:** Inline JSDoc comments in route handlers
- **Stripe Docs:** <https://stripe.com/docs/api>
- **Next.js Docs:** <https://nextjs.org/docs>
- **Prisma Docs:** <https://www.prisma.io/docs>

Quick Reference Commands

Start development

```
npm run dev
```

Database operations

```
npx prisma db push && npx prisma generate
```


Generate test data

node scripts/generate-500-talents.mjs

Run tests

npm run test

Build for production

npm run build && npm run start

Example Prompts for GitHub Copilot

Creating API Routes:

"In 3yases, create /api/subscribe/route.ts that generates a Stripe Checkout session for the Standard plan (£10/6mo). Include auth verification, Zod validation, and proper error handling."

Adding Feature Gates:

"Add subscription check to /dashboard/portfolio/edit - redirect to /pricing if user doesn't have ACTIVE subscription status. Use middleware pattern."

Database Queries:

"Write a Prisma query to get all TalentProfiles with ACTIVE subscriptions, including user data and portfolio count. Add pagination (20 per page) and sort by rating descending."

Component Updates:

"Create a <SubscriptionBadge> client component that shows 'Standard' with teal styling if subscription.status === 'ACTIVE', or 'Upgrade' button linking to /pricing if null. Use Tailwind and next-intl for translations."

Security Implementation:

"Add rate limiting middleware to /api/upload route - max 10 uploads per hour per user. Use in-memory store for dev, document Upstash integration for production."